Assignment - 2: simplement ceeoree control protocol

→ to encode & decode original data and help of parity bits.
demonstrate the use of ceeoree control ~~packets~~ protocols.

theory:

- types of errors:

1. single bit error:
occurs when only one bit changes during transmition

original : 10101
recieved : 10001

2. burst ceerroe:
when 2 or more ceeoree consicutive bits are altered.

original : 10110110
recieved : 10001010

- concept of parity bits:
they are adtional bits added to the original data to detect & correct errors

Sundaram®
FOR EDUCATIONAL USE

cin hamming code : m: data bits
r: parity bits.

$$2^r \geq m + r + 1.$$

- channing code example :

1011

place bits

posⁿ	1	2	3	4	5	6	7
bit	P1	P2	1	P4	0	1	1

P1 = 0
P2 = 1
P4 = 0

find code: 0110011

observation :

channing code algo successfully implemented to perform error correction & detection. Parity bits are calculated and added to the original data. At the receivers side if the syndrome was zero, no error else position of the bit is calc & corrected.

Sundaram
FOR EDUCATIONAL USE

FAQ's.

14

flow control	error detection
controls the rate of data transmission b/w sender & receiver	detects & corrects error in transmitted data
prevents receiver buffer overflow	ensures data integrity & reliability.
concerned w/ transmission speed	concerned w/ transmission accuracy.
eg. sliding window	eg. hamming code

25 A) forward error correction:

in FEC, bits are added to the transmitted data so that the user can detect & correct the error without a re transmission.

adv: no retransmission req.
suitable for realtime communication

disadv: increases data overhead due to additional parity bits

B) Automatic Repeat request:

In it the receivers detects

FOR EDUCATIONAL USE

error errors using methods like CRC. If an error is found, the receiver request sender to send the data.

adv: high accuracy
suitable for reliable communication

disadv: increases delay

8/8/24

ASSIGNMENT 2

```
binary = input("Enter a 4-bit binary number: ")

if len(binary) == 4 and binary[0] != '0'and all(bit in "01" for bit in binary):

    m = len(binary)

    r = 0
    while (2 ** r) < (m + r + 1):
        r += 1

    n = m + r

    hamming = ['0'] * (n + 1)

    j = 0
    for i in range(1, n + 1):
        if i & (i - 1) == 0:
            continue
        hamming[i] = binary[j]
        j += 1

    for i in range(r):
        p = 2 ** i
        parity = 0

        for j in range(1, n + 1):
            if j & p:
```

```
            parity ^= int(hamming[j])

        hamming[p] = str(parity)

    print("Hamming Code:", "".join(hamming[1:]))

else:
    print("Not valid")
```

```
29

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\CollegeTB\CN\LAB> & C:\Users\Lenovo\AppData\Local\Python\pythoncore-3.14-64\python.exe d:/CollegeTB/CN/LAB/2.py
Enter a 4-bit binary number: 1001
Hamming Code: 0011001
PS D:\CollegeTB\CN\LAB> & C:\Users\Lenovo\AppData\Local\Python\pythoncore-3.14-64\python.exe d:/CollegeTB/CN/LAB/2.py
Enter a 4-bit binary number: 1111
Hamming Code: 1111111
PS D:\CollegeTB\CN\LAB> & C:\Users\Lenovo\AppData\Local\Python\pythoncore-3.14-64\python.exe d:/CollegeTB/CN/LAB/2.py
Enter a 4-bit binary number: 1010
Hamming Code: 1011010
PS D:\CollegeTB\CN\LAB>
```

13/8/25